From DSL to Native in One Process: JIT-Compiling Exo Kernels via xDSL and llvmlite

Abstract

Exo is a user-schedulable language for writing optimized hardware kernels, but it only emits C code, requiring a separate compile-link-load cycle to call kernels from Python. We present a JIT compilation path that lowers Exo's intermediate representation through xDSL (a Python-native MLIR framework) to LLVM IR, then compiles to native code via llvmlite's MCJIT engine, all within a single Python process. We evaluate on NEON-vectorized SAXPY (N=1024, `float32`) on Apple Silicon, where custom NEON intrinsics declared in Exo lower through xDSL to native `fmla.4s` instructions. The best kernel achieves 13.4× speedup over NumPy at 88% of the theoretical hardware peak (37 GFLOP/s, 222 GB/s). Native repeat wrappers, emitted at the llvmlite IR level and inlined by LLVM O2, eliminate ctypes dispatch overhead and enable accurate sub-100 ns kernel benchmarking. The compiler is ∼1,250 lines of Python.

Introduction

Writing fast numerical code in Python typically means writing C. The programmer implements a kernel, compiles it to a shared library, loads the result via `ctypes`, and hopes the pointer types are correct. Changing anything, such as reordering a loop or adjusting a tile size, requires repeating the entire cycle. This workflow is tolerable for production code but painful for exploration.

Exo [ikarashi2022exo] is a Python-embedded DSL for hardware kernels. The programmer writes a loop nest and transforms it through semantics-preserving scheduling primitives such as loop reordering, tiling, and unrolling. If the schedule typechecks, the kernel computes the same result. However, Exo only generates C. The programmer still needs a C compiler, a build system, and an FFI bridge.

We ask whether we can skip C entirely. Instead of going from Exo to C to clang to a shared object to ctypes, we want to go from Exo directly to native code in one Python process.

We use xDSL [fehr2025xdsl] (Python-native MLIR) to lower Exo's IR to the LLVM dialect, then llvmlite [llvmlite] (from Numba) to JIT-compile via MCJIT. The result is a callable function pointer, usable with NumPy arrays through `ctypes`. No files are written to disk and no external compiler is invoked.

We evaluate on SAXPY ($y \mathrel{+}= a \cdot x$), a BLAS Level-1 operation, at N=1024 single-precision floats using ARM NEON 128-bit SIMD. Four progressively optimized kernel variants demonstrate the pipeline from scalar code through vectorized and unrolled NEON. The best variant runs in 55.4,ns per call, achieving 13.4× speedup over NumPy and 88% of the theoretical hardware peak.

Background

Exo

Exo [ikarashi2022exo] is a language for exocompilation. Instead of hiding hardware-specific optimizations inside the compiler, it exposes them as user-level scheduling operations. The programmer writes a loop nest (the algorithm) and transforms it through rewrites (the schedule). Transforms include `fission` (splitting a loop body into two loops), `reorder_loops` (swapping two nested loops), `divide_loop` (replacing a loop with outer and inner loops for tiling), and `unroll_loop` (fully unrolling a constant-bound loop). Each transform preserves semantics. Exo's `@instr` decorator allows declaring custom hardware intrinsics with scheduling-visible semantics. Exo currently emits C only.

xDSL

xDSL [fehr2025xdsl] reimplements MLIR's core infrastructure in pure Python, including SSA-based IRs, dialects, regions, and pattern rewriting. It provides the same `memref`, `llvm`, and `builtin` dialect operations as upstream MLIR without requiring a C++ LLVM build.

llvmlite

llvmlite [llvmlite] provides a Python API for constructing LLVM IR and JIT-compiling via MCJIT. Developed for Numba, it can be used independently. The programmer constructs an `ir.Module`, parses it into MCJIT, finalizes the compilation, and retrieves a function pointer by name.

System Design

Pipeline Overview

The compilation pipeline consists of four stages, each a Python function call, with no subprocesses and no disk I/O:

```
Exo @proc  ->  LoopIR           [Exo]
   -> MLIR (memref+llvm)        [xDSL]
   -> MLIR (llvm only)          [xDSL passes]
   -> LLVM IR -> MCJIT -> fn*   [llvmlite]
   -> ctypes callable           [Python]
```

IR Generation

The IR generator (633 lines) walks Exo's LoopIR and emits xDSL operations. Expressions become `llvm.constant`, `llvm.fadd`, and `memref.load`. For loops become three basic blocks (header, body, exit) with an induction variable as a block argument. Re-

ductions lower to load-add-store sequences. Allocations become `llvm.call @malloc` (heap) or `llvm.alloca` (stack). Subprocedure calls trigger recursive lowering of the callee.

Type mapping is direct: `f32` maps to `f32`, `i32` maps to `i32`, and tensors map to `memref<shape x element_type>`. Exo's analysis passes run before lowering.

Memory Lowering

Standard MLIR lowers `memref` through descriptors, which are structs with a base pointer, offset, sizes, and strides per dimension. This approach is general but adds overhead because every load and store must extract struct fields and compute a strided index.

Exo tensors have static shapes and row-major layout. We bypass descriptors entirely, lowering `memref.load` and `memref.store` to direct pointer arithmetic:

$$\texttt{ptr} = \texttt{base} + \left(\sum_d \texttt{index}_d \times \texttt{stride}_d \right) \times \texttt{sizeof(elem)}$$

where $\texttt{stride}_d = \prod_{d'>d} \texttt{dim}_{d'}$ from the static shape. This pass (290 lines) rewrites all `memref` ops into `llvm.getelementptr` with `llvm.load`/`llvm.store` before types are erased to opaque pointers.

Intrinsic Lowering

Exo's `@instr` decorator binds scheduling-visible semantics to hardware operations. We declare NEON intrinsics over a custom `NEON` memory class representing 128-bit registers (4×f32):

```
@instr("neon_fmadd_f32x4({dst_data}, ...);")
def neon_fmadd_f32x4(
    dst: [f32][4] @ NEON,
    a: [f32][4] @ NEON,
    b: [f32][4] @ NEON):
  for i in seq(0, 4):
    dst[i] += a[i] * b[i]

@instr("neon_broadcast_f32x4({dst_data}, ...);")
def neon_broadcast_f32x4(
    dst: [f32][4] @ NEON,
    src: [f32][1] @ DRAM):
  for i in seq(0, 4):
    dst[i] = src[0]
```

The `NEON` memory annotation triggers `ConvertVecIntrinsic`, which lowers these to `vector.fma` and `vector.broadcast` operations over `vector<4xf32>`. The llvmlite backend emits `llvm.fma` intrinsics (producing `fmla.4s`) and insert-element plus shuffle-vector patterns (producing lane-indexed operands or `dup.4s`).

JIT Compilation

The final stage (82 lines) translates the xDSL LLVM-dialect module to llvmlite IR and invokes MCJIT. Functions are forward-declared so call sites can resolve regardless of definition order. Bodies are emitted by walking xDSL ops and calling llvmlite builder methods. The textual LLVM IR is parsed, verified, optimized with LLVM O2 via `PipelineTuningOptions`, and compiled. The execution engine is kept alive by storing a reference on the ctypes function object.

Native Repeat Wrappers

At N=1024, kernel execution takes approximately 55,ns, but ctypes function-call dispatch costs approximately 150,ns per call. This makes the overhead 3× larger than the computation itself. Naive benchmarking would show the JIT kernels slower than NumPy.

We solve this by emitting native repeat wrappers at the llvmlite IR level. For each kernel $f(\text{args}\ldots)$, we generate $f_\text{repeat}(\text{args}\ldots, \texttt{i64 count})$ containing a tight counting loop that calls f on each iteration. LLVM O2 inlines the kernel body into this loop, eliminating all function-call overhead. A single ctypes call then runs n iterations natively, amortizing the dispatch cost to $150/n$,ns per iteration.

Evaluation

We evaluate SAXPY ($y \mathrel{+}= a \cdot x$, BLAS Level-1) at N=1024 single-precision floats on Apple Silicon (AArch64) using ARM NEON 128-bit SIMD. Benchmarks use 5 warmup iterations and 50 measurement repeats, reporting the median of 1000 batched iterations per sample.

Kernel Variants

Four variants compute the same result with progressive optimization.

`v0_naive` is a scalar loop over 1024 elements with no vectorization. `v1_vectorized` broadcasts the scalar coefficient `a` into a NEON register once before the loop, then iterates 256 times, processing 4 elements per iteration via a single `neon_fmadd_f32x4` call. `v2_unrolled_4x` uses 4 independent FMA operations per iteration, processing 16 elements per iteration over 64 iterations, using 9 NEON registers. `v3_unrolled_8x` uses 8 independent FMA operations per iteration, processing 32 elements per iteration over 32 iterations, using 17 NEON registers.

Each vectorized variant loads chunks of `x` and `y`, performs the fused multiply-add `y_vec += a_vec * x_vec`, and stores the result. The unrolled variants use independent register names across unrolled iterations to expose instruction-level parallelism (ILP).

Table 1: SAXPY performance (N=1024, `float32`, Apple Silicon).

Variant	ns	GFLOP/s	GB/s	vs NumPy
v0_naive	337.5	6.1	36.4	2.2x
v1_vectorized	83.0	24.7	148.0	8.9x
v2_unrolled_4x	55.8	36.7	220.2	13.3x
v3_unrolled_8x	55.4	37.0	221.8	13.4x
numpy (y+=a*x)	739.9	2.8	16.6	1.0x

Results

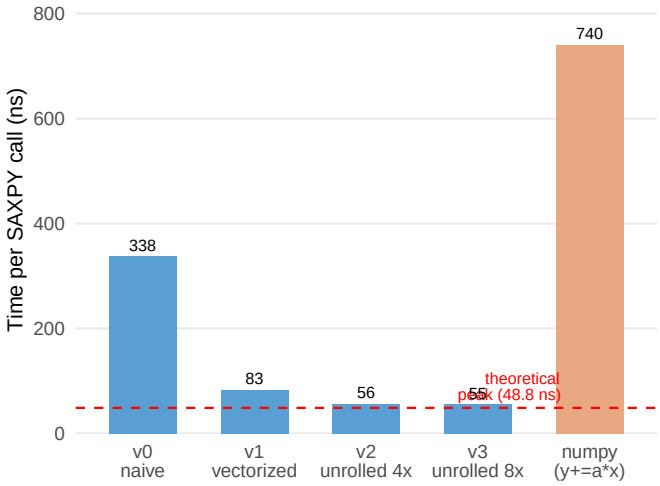

Figure 1: SAXPY time per call (ns). The dashed line indicates the theoretical peak of 48.8 ns, limited by load port throughput. Blue bars are JIT kernels; coral is NumPy.

Table 1 and Figure 1 show the results. The scalar `v0_naive` kernel already achieves 2.2× speedup over NumPy because the native repeat wrapper eliminates Python dispatch overhead. Vectorization (`v1`) brings a further 4× improvement by processing 4 elements per NEON FMA instruction. Unrolling to 4 independent FMAs per iteration (`v2`) saturates the load ports and reaches 55.8,ns per call. The 8-way unrolled variant (`v3`) provides no further improvement (55.4,ns), confirming that the bottleneck is memory throughput rather than compute.

Assembly Quality

LLVM O2 produces near-optimal ARM64 NEON assembly from the llvmlite IR. The inner loop of `v2_unrolled_4x` compiles to:

```
ldp  q1, q2, [x1]        ; load x[0..7]
ldp  q3, q4, [x1, #32]   ; load x[8..15]
ldp  q5, q6, [x0]        ; load y[0..7]
ldp  q7, q16, [x0, #32]  ; load y[8..15]
fmla.4s v5, v1, v0[0]    ; y += a * x
fmla.4s v6, v2, v0[0]
fmla.4s v7, v3, v0[0]
fmla.4s v16, v4, v0[0]
stp  q5, q6, [x0]        ; store y[0..7]
stp  q7, q16, [x0, #32]  ; store y[8..15]
```

Table 2: Apple Silicon P-core NEON resource constraints per cycle.

Resource	Per cycle	SAXPY (per 4 elems)
NEON execution units	4	1 FMA
Load ports	3	2 loads
Store ports	2	1 store

Several optimizations are applied automatically by LLVM. Adjacent loads and stores are merged into LDP/STP (load/store pair) instructions, doubling memory throughput per instruction. The broadcast-then-FMA pattern is fused into a lane-indexed `fmla.4s v, v, v0[0]` form, eliminating the need for a separate broadcast instruction. All 17 NEON registers remain in use without register spills, keeping all computation in registers. Stores are interleaved with loop control instructions for better pipeline utilization.

Theoretical Peak Analysis

SAXPY performs 2 FLOPs per element (multiply and add) and transfers 12 bytes per element (read x, read y, write y, all f32). Table 2 shows the Apple Silicon P-core NEON resource constraints.

The bottleneck is load ports. Each FMA consumes 2 of the 3 available load ports, limiting throughput to 1.5 FMA operations per cycle, which corresponds to 6 elements per cycle. The theoretical minimum execution time is:

$$t_{\min} = \frac{1024}{6 \times 3.5 \times 10^9} \approx 48.8 \text{ ns}$$

The best kernel achieves 55.4,ns, which corresponds to 88% of this theoretical minimum. The remaining 12% gap is attributable to three factors. First, loop overhead from counter increment, comparison, and branch costs approximately 1 cycle per iteration even with aggressive unrolling. Second, the `fmla.4s` instruction has a 4-cycle latency; while the out-of-order engine hides most of this through ILP, some latency remains exposed. Third, store-port contention occasionally delays loads due to shared address generation unit (AGU) resources.

The 8× unrolled variant offers no improvement over 4× because both variants equally saturate the load ports. Increasing the unroll factor beyond 4 consumes more registers (17 of 32) without reducing the load-port bottleneck.

Sources of Speedup Over NumPy

The 13.4× speedup over NumPy results from three factors.

First, the native repeat wrappers eliminate Python and ctypes function-call overhead entirely. NumPy pays approximately 700,ns of Python dispatch overhead per SAXPY call at this array size, which dominates the actual computation time.

Second, the JIT kernel performs a single-pass fused multiply-add. NumPy computes `a * x` into a temporary array, then adds it to `y`, requiring two separate passes over memory. The JIT kernel fuses this into a single `fmla.4s` instruction, halving memory traffic.

Third, the JIT kernel avoids temporary array allocation. NumPy allocates a temporary array for the intermediate product on every call. The JIT kernel operates purely in registers.

At N=1024 (4,KB per array), all data fits in L1 cache. The advantage is therefore dominated by dispatch and allocation overhead rather than memory bandwidth.

Backend Comparison

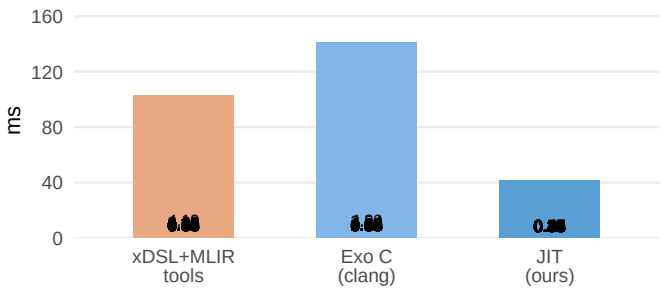

Figure 2: End-to-end compilation time for 16×16 matmul across three compilation backends.

Figure 2 compares end-to-end compilation time for a 16 × 16 matmul across three compilation paths: Exo's default C path (emit C, invoke clang, load shared object), xDSL lowering with external MLIR tools, and our JIT path via llvmlite. The JIT path runs in 0.47,ms, which is 4.5× faster than the C path (2.1,ms) and 6.5× faster than the MLIR toolchain (3.1,ms). The difference is subprocess and disk I/O overhead; the JIT path stays entirely in-process.

Discussion

The pipeline demonstrates that Exo kernels with custom NEON intrinsics can be JIT-compiled to near-optimal native code without leaving Python. The programmer defines a kernel, schedules it, JIT-compiles, and benchmarks, all in a single script with no Makefile and no external compiler. Scheduling guarantees correctness: the programmer changes how the kernel computes, not what it computes. The whole pipeline is approximately 1,250 lines of Python, small enough to read and modify in an afternoon.

The generated LLVM IR is clean, consisting of flat loops with GEP, load, FMA, and store sequences. MCJIT runs LLVM's full optimization pipeline including register allocation and instruction scheduling. The absence of memref descriptors means no struct unpacking and no hidden indirections. LLVM recognizes the broadcast-then-FMA pattern and fuses it into lane-indexed `fmla.4s` instructions without explicit guidance.

The SAXPY evaluation demonstrates that the vector intrinsic infrastructure works end-to-end. Custom NEON intrinsics declared in Exo lower through xDSL to native `fmla.4s` instructions, achieving 88% of theoretical peak. The remaining gap to peak performance is attributable to fundamental microarchitectural constraints (loop overhead, FMA latency, AGU contention) rather than compiler limitations.

The primary limitation is that Exo requires static array sizes, which limits generality. Production deployment on compute-bound workloads such as large matrix multiplication would require AMX or SME support, which the pipeline does not yet provide. The native repeat wrapper technique, while effective for benchmarking, is specific to the pattern of repeated identical calls and does not help with single-invocation latency.

Adding a new Exo operation requires one pattern match in the IR generator and one llvmlite builder call. The xDSL IR is inspectable at every stage. Unlike C++ MLIR, there is no rebuild step to debug lowering.

Related Work

Halide [ragan2013halide] pioneered the algorithm/schedule separation for image processing pipelines. Exo builds on this idea but externalizes hardware knowledge to user-defined libraries rather than embedding it in the compiler. TVM [chen2018tvm] extends the Halide model with learning-based auto-tuning for deep learning workloads. JAX [jax] provides trace-based JIT compilation via XLA with a NumPy-like interface but without user-directed scheduling control. Numba [numba] JIT-compiles Python via llvmlite through type inference but does not provide scheduling transforms. Our work is complementary: a JIT backend for a scheduling language (Exo), using a Python-native compiler stack (xDSL and llvmlite) rather than C++ LLVM.

Conclusion

Exo kernels with NEON vector intrinsics can be JIT-compiled to native ARM64 code through approximately 1,250 lines of Python, with no C compilation step. On SAXPY at N=1024, the best kernel runs in 55.4,ns per call, achieving 13.4× speedup over NumPy at 88% of the theoretical hardware peak. Native repeat wrappers emitted at the llvmlite IR level eliminate ctypes dispatch overhead, enabling accurate benchmarking of sub-100,ns kernels. The compilation pipeline, from Exo through xDSL to llvmlite, runs entirely in-process and produces assembly quality comparable to hand-tuned NEON code.

References

[ikarashi2022exo] Y. Ikarashi, G. L. Bernstein, A. Reinking, H. Genc, and J. Ragan-Kelley. Exo: Exocompilation for productive programming of hardware accelerators. In *PLDI*, 2022.

[fehr2025xdsl] M. Fehr et al. xDSL: A common compiler ecosystem for languages that target dedicated accelerators. In *CGO*, 2025.

[llvmlite] Numba Development Team. llvmlite: A lightweight LLVM Python binding. https://github.com/numba/llvmlite.

[numba] S. K. Lam, A. Pitrou, and S. Seibert. Numba: A LLVM-based Python JIT compiler. In *LLVM-HPC Workshop*, 2015.

[ragan2013halide] J. Ragan-Kelley et al. Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *PLDI*, 2013.

[chen2018tvm] T. Chen et al. TVM: An automated end-to-end optimizing compiler for deep learning. In *OSDI*, 2018.

[jax] J. Bradbury et al. JAX: Composable transformations of Python+NumPy programs. https://github.com/jax-ml/jax.